Discussion 4 – Stacks Queues and Deques

Discussion Topic:

How do stacks, queues, and deques solve different types of problems in software applications, and what properties make each data structure appropriate for specific use cases? Provide a real-world example to support your analysis.

My Post:

Hello Class,

Stacks, queues, and deques are data structures with linear organization; however, they differ in how they handle data processing.

A stack employs a Last-In, First-Out (LIFO) approach to data management. It implements a functionality where the most recently added item is the first one removed. This approach makes stacks useful when processing the newest item first is required; in programming applications, stacks are commonly used to store return addresses during function calls, making them well suited for managing nested program execution (CSU Global, n.d.). Stacks usually use arrays, which are contiguous blocks of memory; within the array, each element has an index denoting the element's location in memory.

A queue, on the other hand, employs a First-In, First-Out (FIFO) approach to data management, meaning the first item added, the oldest item, is the first one removed. This approach is similar to a line of people waiting at a grocery store, where the first person in line is the first person to be helped (CSU Global, n.d.). In other words, queues are useful when data must leave in the same order it arrived (TutorialsPoint, n.d.). Usually, queues use singly linked lists for implementation, that is, each node in the list points to the next node in the list. In practice, queues are useful for applications such as print spooling, customer service systems, task scheduling, or message processing.

Instead of a singly linked list, a deque usually uses a doubly linked list, which is more flexible as items can be inserted and removed from both the front and the back, that is, each node in the list points to the next node in the list and the previous node in the list (Lysecky & Vahid, 2016). This approach is more flexible than either a stack or a standard queue, making deques useful for applications that require processing at either end of a dataset, such as browser history, sliding-window algorithms, job schedulers with priority adjustments, or palindrome checking.

A real-world example that may use all three of these data structures is a customer support software. A stack can be used to track an agent's most recent actions, such as recently opened case records or undoing the latest change to a ticket. A deque can be used to handle both urgent and non-urgent incoming tickets, as high-priority tickets can be inserted at the front of the deque while regular tickets are placed at the back, before being assigned to an agent. A queue can be used by an agent to handle tickets assigned to her/him in the order in which they were assigned.

-Alex

References

CSU Global. (n.d.). *Module 4: Lists, stacks, and queues.* [Interactive lecture]. CSC506 - Design and Analysis of Algorithms. Canvas.

TutorialsPoint. (n.d.). *Queue data structure*. TutorialsPoint. https://www.tutorialspoint.com/data_structures_algorithms/dsa_queue.htm

Lysecky, R., & Vahid, F. (2019). Module 4: Lists, stacks, and queues. *Data structures essentials: Pseudocode with Python examples.* zyBooks